INCIDENT + RUNBOOK · 1 SEPTEMBER 2026

Stockiha Launch Runbook

The app would not start: PostgreSQL on port 5433 refused every connection. The cause was a single leftover lock file — not a broken database, and not broken code. Here is how to launch Stockiha properly, how to clear this exact fault if it returns, and what an engineer or AI agent needs to know to diagnose it from scratch.

BRANCH	COMMIT	MIGRATIONS
task/ws-d-product-inventory	376627a — WS-D-4	PASS · 139 applied
APP WINDOW	**DATABASE**	**DOWNTIME**
Open · PID 10060	127.0.0.1:5433	~21 minutes

PART ONE · FOR EVERYDAY USE

How to start Stockiha

Stockiha needs three things awake before you can see the login screen: its own private database, its up-to-date data structure, and the app itself. One script does all three in order. You never need to run them by hand.

01 **Open PowerShell in the project folder**

Press the Windows key, type **PowerShell**, open it, then paste this line and press Enter.

```
cd C:\Users\Perfetto\Desktop\Stockiha-Part02-Test
```

02 **Run the launcher**

This is the only command you need. It starts the database, updates it, builds the app, and opens the window.

```
.\run.bat
```

03 **Wait for the window — about 3 to 5 minutes**

The text scrolling past is normal. Watch for these five markers in order; the last one means the app is on its way to the screen.

```
[1/5] PostgreSQL is accepting connections on port 5433
[2/5] Database migrations: PASS
[4/5] Credentials loaded
      Building frontend production bundle
[5/5] Launching Tauri dev window
```

04 **Leave the black window open**
That console window is running Stockiha. Closing it closes the app. When you are finished for the day, close the Stockiha window first, then the console.

> **NEVER START IT THIS WAY**
> Do not run `npm run tauri dev` on its own. It skips the database startup, skips the migration step, and gives you an app with no data behind it — which is exactly how this incident began. A session started that way was already running here, and it had no database at all.

PART TWO · IF IT HAPPENS AGAIN

Fixing “PostgreSQL is not running on port 5433”

You will know it is this fault when `.\run.bat` stops almost immediately, at step 1 of 5, with wording close to this:

```
[1/5] Ensuring PostgreSQL is accepting connections on port 5433...
Starting PostgreSQL cluster on port 5433...
pg_ctl: another server might be running; trying to start server anyway
 ERROR: Failed to ensure PostgreSQL is running on port 5433.
```

“Another server might be running” is misleading. In this case nothing was running. The database had been shut down uncleanly earlier in the day and left behind a note saying “I am starting up” — a lock file that was never cleaned away. The next start-up read that note, believed it, and refused. Deleting the note is the fix. Run these five commands in order, in PowerShell.

01 **Confirm the database really is stopped**
This must answer `pg_ctl: no server running`. If it says the server is running, stop here — the problem is something else, and the steps below would be unsafe.

```
& 'C:\Program Files\PostgreSQL\18\bin\pg_ctl.exe' status -D "$env:LOCALAPPDATA\Stockiha\r8-acceptance\data-55433"
```

02 **Remove the leftover lock file**
Only this one file. It is a marker, not data — nothing in your database is lost.

```
Remove-Item "$env:LOCALAPPDATA\Stockiha\r8-acceptance\data-55433\postmaster.pid" -Force
```

03 **Start the database**

```
& 'C:\Program Files\PostgreSQL\18\bin\pg_ctl.exe' start -D "$env:LOCALAPPDATA\Stockiha\r8-acceptance\data-55433" -o "-p 5433" -l "$env:LOCALAPPDATA\Stockiha\r8-acceptance\postgres-5433-launch.log"
```

04 **Wait until it answers “accepting connections”**

Run this every minute or so. After an unclean shutdown the database repairs itself first, which took about **9 minutes** here. Until it finishes you will see `rejecting connections` — that is progress, not failure. Do not restart it, and do not delete anything else while you wait.

```
& 'C:\Program Files\PostgreSQL\18\bin\pg_isready.exe' -h 127.0.0.1 -p 5433 -U stockiha_admin
```

05 **Start the app normally**

```
.\run.bat
```

TWO THINGS NEVER TO DO

Never delete the data-55433 folder to clear a lock. That folder is the database itself — every product, sale, and journal entry lives there. The old troubleshooting note suggesting a wipe applies only to a database you are willing to lose entirely.

Never stop the Windows service postgresql-x64-18. That is the separate PostgreSQL on port 5432 and it is not what Stockiha uses. Stopping it does nothing for this fault.

HARMLESS MESSAGE

Once the app opens you may see a warning about `STOCKIHA_DEV_DATABASE_URL` overriding the local key. That is the launcher doing its job. It only matters if the address shown is *not* `127.0.0.1:5433/stockiha_acceptance`.

PART THREE · FOR ENGINEERS AND AI AGENTS

Incident analysis: stale postmaster.pid on the 5433 cluster

Symptom and true root cause

`scripts\ensure-postgres.ps1` aborted `run.bat` at step 1. `pg_ctl start` emitted *“another server might be running; trying to start server anyway”*, then the 15-second `pg_isready` poll expired.

The data directory `%LOCALAPPDATA%\Stockiha\r8-acceptance\data-55433` held a `postmaster.pid` naming PID 4072 with state `starting`, written when the cluster died mid-startup. The cluster's last clean checkpoint was `2026-09-01 14:44:20`. PID 4072 no longer existed, and `pg_ctl status -D` returned `no server running` — together, conclusive proof the pid file was stale rather than held.

DIAGNOSTIC ORDER THAT MATTERS

`pg_ctl status -D <datadir>` is the authority, not the pid file and not the `pg_ctl start` warning text. Removing a `postmaster.pid` that a live postmaster still owns can produce two postmasters over one data directory and corrupt it. Verify absence twice — no such PID, and status reporting no server — before deleting.

Second, independent fault

A `tauri dev` process tree had been running since 20:29 (Vite bound on 1420, `stockiha-backend.exe` alive) with no run log under `logs\` and no `STOCKIHA_DEV_DATABASE_URL` — a bare `npm run tauri dev`, which bypasses cluster startup, migrations, and credential resolution. `run.bat`'s own `cleanup-dev-processes.ps1` reaped it on the next launch. Absence of a timestamped `logs\stockiha-*.log` is the reliable tell that a running session did not come from the launcher.

Recovery sequence, with evidence

TIME	EVENT	EVIDENCE
14:44:20	Cluster's last clean checkpoint	`database system was interrupted; last known up at …`
20:29	Bare tauri dev session started	No run log; no DB URL in environment
20:37:39	run.bat aborts at step 1	`ERROR: Failed to ensure PostgreSQL is running`
20:38:29	Stale pid removed; cluster started	`listening on IPv4 address “127.0.0.1”, port 5433`
20:38–20:39	Crash recovery: datadir fsync, then redo	`redo done at 0/59724878`
20:39–20:47	End-of-recovery checkpoint, slowed by WAL locks	`could not rename file “pg_wal/…”: Permission denied`
20:47	Cluster open	`127.0.0.1:5433 - accepting connections`
20:47:32	run.bat rerun	`Database migrations: PASS · 139 applied`
20:50:34	Window up on WS-D-4	`stockiha-backend PID 10060, title Stockiha`

On the WAL rename failures

Recovery logged repeated `could not rename file “pg_wal/000000010000000000000004X”: Permission denied` while recycling segments. On Windows this is an external handle on the WAL file — realtime antivirus scanning is the usual holder. PostgreSQL treats it as soft: the segment is not recycled and the checkpoint proceeds. It is a throughput problem (roughly 30 s per affected segment), not corruption, and it does not by itself justify intervention. Adding an antivirus exclusion for the data directory would remove the stall, but it changes a system security setting and is the operator's decision, not an agent's.

Rules for an agent handling this next time

- **Establish how the app was started before diagnosing it.** Check logs\stockiha-*.log for a run matching the process start time; a running tauri dev with no such log is not a valid baseline and its failures prove nothing about the code.
- **Prove the negative before deleting a lock.** Require both Get-Process -Id <pid-from-file> returning nothing and pg_ctl status -D returning no server running.
- **Distinguish “rejecting” from “refused”.** rejecting connections means a live postmaster is still in recovery — wait. Connection refused (os error 10061) means nothing is listening — start the cluster. These have opposite remedies.

- **Never treat recovery time as a hang.** Poll pg_isready in a bounded loop and read postgres-5433-launch.log for forward progress (redo LSN advancing, WAL segment numbers climbing) rather than restarting the postmaster.
- **Never widen the blast radius.** The 5432 service, the data-55433 directory, and Defender settings are all out of scope for a stale lock file.
- **Report only what was observed.** A launched process is not a working app; MainWindowTitle plus the run log's database target is the minimum evidence that the window is up and pointed at the right cluster.

Verified state at close

> **CONFIRMED**
>
> Cluster accepting connections on 127.0.0.1:5433 · 139 migrations applied · frontend bundle built in 7.36 s · stockiha-backend compiled in 2 m 14 s · window open as PID 10060 against stockiha_acceptance, running branch task/ws-d-product-inventory at 376627a (WS-D-4).

Not verified: no WS-D screen was exercised in this session. Catalogue and product-list behaviour on `list_products_v2` remains untested at the UI level.
